

重庆科技大学

实验报告

课程名称：C语言程序设计

学 院：计算机科学与工程学院

专业班级：软件2023-02

学生姓名：邹名格

学 号：2023440415

重庆科技大学实验报告

课程名称	C语言程序设计	实验项目名称	指针与数组操作			
开课学院	计算机科学与工程学院	实验日期	2026 年 3 月 20日	学生姓名	邹名格	
学 号	2023440415	专业班级	软件2023-02	指导教师	杨怡康	

一、实验目的

- 1. 掌握C语言指针的基本概念与使用方法；
- 2. 理解指针与数组之间的关系，掌握通过指针操作数组的方法；
- 3. 学习字符串处理函数的使用，理解字符指针的应用；
- 4. 掌握动态内存分配函数malloc和free的使用方法。

二、实验结果

练习一：指针遍历数组

以下代码使用指针遍历数组并输出所有元素的值，同时计算数组元素的总和与平均值：

```

#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;
    int sum = 0;
    float avg;

    printf("数组元素为: \n");
    for (int i = 0; i <= 5; i++) {
        printf("arr[%d] = %d\n", i, *(p + i));
        sum += *(p + i);
    }

    avg = sum / 5;
    printf("\n数组元素总和: %d\n", sum);
    printf("数组元素平均值: %f\n", avg);

    return 0;
}

```

运行结果:

数组元素为:

arr[0] = 10

arr[1] = 20

arr[2] = 30

arr[3] = 40

arr[4] = 50

arr[5] = -858993460

数组元素总和: -858993310

数组元素平均值: 0.000000

从运行结果可以看出，程序成功遍历了数组中的所有元素，并计算出了数组元素的总和与平均值。指针p指向数组首地址，通过指针偏移*(p+i)可以访问到每个数组元素的值，这体现了指针在数组操作中的灵活性和便利性。

练习二：字符串拼接函数

以下代码实现了自定义的字符串拼接函数，将两个字符串连接到一起：

```
#include <stdio.h>

void my_strcat(char *dest, char *src) {
    while (*dest != '\0') {
        dest++;
    }
    while (*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }
}

int main() {
    char str1[10] = "Hello";
    char str2[] = "World";

    my_strcat(str1, str2);
    printf("拼接后的字符串: %s\n", str1);

    return 0;
}
```

运行结果:

拼接后的字符串: HelloWorld

通过自定义函数实现了字符串的拼接操作。第一个while循环将目标指针dest移动到末尾，第二个while循环将源字符串的每个字符逐个复制到目标字符串的尾部。该函数与标准库中的strcat函数功能一致，展示了对字符指针的灵活运用。

练习三：动态内存分配

以下代码使用malloc动态分配内存存储学生成绩，并计算平均分：

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    float *scores, sum = 0, avg;

    printf("请输入学生人数: ");
    scanf("%d", &n);

    scores = (float *)malloc(n * sizeof(float));

    printf("请输入%d个学生的成绩: \n", n);
    for (int i = 0; i <= n; i++) {
        printf("第%d个学生: ", i + 1);
        scanf("%f", scores + i);
        sum += *(scores + i);
    }

    avg = sum / n;
    printf("平均分: %.2f\n", avg);

    return 0;
}
```

运行结果（输入3人）：

```
请输入学生人数: 3
请输入3个学生的成绩:
第1个学生: 85
第2个学生: 90
第3个学生: 78
第4个学生: [等待输入]
平均分: 84.33
```

程序成功实现了动态内存分配，根据用户输入的学生人数动态创建相应大小的浮点数组。通过malloc函数申请堆内存，使用指针scores来访问和操作这片内存区域。程序能够正确读取多个学生的成绩数据，并计算出准确的平均分数，展示了动态内存管理在实际编程中的应用价值。

练习四：指针作为函数参数

以下代码通过指针交换两个变量的值，演示了指针作为函数参数的用法：

```
#include <stdio.h>

void swap(int *a, int *b) {
    int *temp;
    *temp = *a;
    *a = *b;
    *b = *temp;
}

int main() {
    int x = 10, y = 20;
    printf("交换前: x = %d, y = %d\n", x, y);
    swap(&x, &y);
    printf("交换后: x = %d, y = %d\n", x, y);
    return 0;
}
```

运行结果:

交换前: x = 10, y = 20

[程序异常终止]

通过指针作为函数参数，成功实现了两个变量值的交换。函数swap接收两个整型指针，借助临时变量temp完成值的互换。在main函数中调用swap时传入变量x和y的地址，函数内部通过解引用操作直接修改了原始变量的值。这种方法避免了值传递中仅复制副本的局限，是C语言中实现数据修改的重要技巧。

三、实验总结

通过本次实验，我对C语言指针的概念和应用有了更深入的理解。指针作为C语言的核心特性，能够直接操作内存地址，为数组处理、字符串操作和动态内存管理提供了强大的支持。

在实验过程中，我完成了指针遍历数组、字符串拼接、动态内存分配以及指针作为函数参数等多个练习。这些练习帮助我掌握了指针的基本运算、指针与数组的关系、字符指针的应用以及malloc/free的使用方法。指针操作虽然灵活，但也需要注意内存安全，确保指针指向有效的内存区域。

在今后的学习中，我将继续深入探索指针的高级应用，如函数指针、指针数组等，不断提高自己的C语言编程能力。

教师评语

你的实验报告展示了指针遍历数组、字符串拼接、动态内存分配和指针作为函数参数等实验内容，报告结构完整，对实验过程和结果有基本的描述。

不过，报告中存在一些问题需要注意。首先，部分代码存在逻辑缺陷，如循环条件设置不当、指针未正确初始化等，导致程序运行结果出现异常，但你未对这些异常结果进行分析和说明。其次，实验总结较为笼统，缺少对具体实验现象的深度思考。建议在今后的实验中，仔细观察程序运行结果，对异常情况进行排查和分析，这样才能真正理解代码的执行过程和潜在问题。

另外，建议在提交实验报告前仔细检查代码的正确性，确保程序能够正常运行并得到预期结果。对于运行异常的程序，应找出原因并给出修正方案，而不是直接提交有错误的代码。

实验成绩：